

Herald

Chaque alerte parvient à la bonne destination — sans syntaxe de commande requise.

Résumé

Herald ingère les événements système et les redistribue de manière fiable vers plusieurs canaux de notification, garantissant que chaque alerte atteint sa cible. Les abonnés interagissent en langage naturel ; Herald déduit l'intention grâce à une approche à trois niveaux (raccourci de commande → inférence d'intention via LLM → repli de clarification).

Description courte

Un système d'ingestion d'événements et de redistribution multi-canaux des notifications. Herald achemine de manière fiable les événements système vers les bonnes destinations, quel que soit le canal de messagerie, et permet aux abonnés de s'exprimer en langage naturel — en résolvant l'intention via un raccourci de commande, une inférence par LLM et un mécanisme de repli par clarification.

Description longue

Herald est l'épine dorsale des notifications, assurant qu'un événement système parvient effectivement là où un humain peut agir — cette couche ingrate mais critique où la plupart des systèmes d'alerte maison échouent en silence. Il ingère les événements et les redistribue de manière fiable vers plusieurs canaux de notification, éliminant les modes de défaillance classiques où une alerte est perdue, mal acheminée vers un canal inactif ou noyée sous le bruit jusqu'à ce qu'il soit trop tard pour réagir. Mais une diffusion fiable n'est que la moitié de l'équation ; l'autre moitié concerne ce qui se passe lorsqu'un humain souhaite répondre. Ici, Herald rejette le compromis habituel où les utilisateurs doivent mémoriser une syntaxe de commande rigide pour interagir avec un bot d'alerte. Les abonnés s'expriment

simplement en langage naturel, et Herald interprète leur intention grâce à une approche méthodique en trois étapes : un raccourci qui reconnaît instantanément les commandes explicites, puis une inférence d'intention basée sur LLM (via le code Claude) pour les messages libres, et enfin un repli de *clarification* qui répond, étiquette et pose une question lorsque l'intention reste ambiguë. Cette échelle « *reconnaître* → *inférer* → *clarifier* » résume à elle seule la philosophie de conception — le cas courant reste instantané et déterministe, le cas flexible est géré par un modèle, et le cas incertain n'est jamais résolu par une supposition aveugle qui déclencherait une mauvaise action.

Herald intègre également la gestion des participants et de l'attribution : une variable d'environnement pour le nom d'utilisateur de l'opérateur (HERALD_<CANAL>_OPERATOR_USERNAME) et un contrat de participation/attribution alimentent les champs `created_by/assigned_to` ainsi que le marquage @ des notifications, afin qu'il soit toujours clair qui a fait quoi et qui est notifié. Sur le plan de la gouvernance, Herald hérite de Helix Constitution en tant que sous-module colocalisé et en suit les règles. Il est également un utilisateur précoce en production de Docs Chain — l'intégralité de son corpus de 66 documents Markdown→HTML/PDF/DOCX est acheminée via les transformations `exec` : de Docs Chain et vérifiée sans erreur. Herald repose principalement sur des outils Shell/Go, avec des spécifications en couches (remplacement V1→V2→V3→V4) et des guides de configuration par canal pour les messageries et les répartiteurs LLM/agents.

Pourquoi nous l'avons conçu

Les alertes échouent discrètement — envoyées vers le mauvais canal, perdues ou exigeant une syntaxe de commande rigide que les utilisateurs ne retiendront pas. Herald a été conçu pour garantir une redistribution fiable et permettre aux utilisateurs de répondre en langage naturel, rendant les notifications à la fois fiables et faciles à traiter.

Contenu

Pourquoi c'est un tournant

Il fusionne deux éléments habituellement achetés séparément – un routage d'événements multi-canaux fiable et une interface en langage naturel – en un seul système où les opérateurs n'ont qu'à parler, et le logiciel interprète leurs intentions. Ce qui le rend digne de confiance en production, c'est précisément son

mécanisme de repli par clarification : un système d'alerte qui préfère demander plutôt que de se tromper est un système que l'on peut effectivement laisser interagir avec des données réelles.

Ce qui est innovant

- Une discipline d'intention à trois niveaux : exécution rapide des commandes → inférence LLM → clarification et demande de confirmation.
- Interaction avec les abonnés en langage naturel (aucune syntaxe de commande à apprendre).
- Contrat d'attribution des participants gérant `created_by/assigned_to` + le marquage par `@`.
- V véritable consommateur Docs Chain (corpus de 66 documents, multi-format, vérifié).

Défis et solutions

- **Intention ambiguë en langage naturel** : résolu par une échelle de reconnaissance/inférence/clarification à trois niveaux, évitant les suppositions aveugles.
- **Diffusion fiable** : résolu par une conception ingestion→diffusion multi-canaux garantissant que les alertes atteignent la bonne destination.
- **Attribution correcte entre les canaux** : résolu grâce à la variable d'environnement du nom d'utilisateur de l'opérateur et au contrat d'attribution des participants.
- **Dérive de la documentation** : résolu en intégrant son corpus documentaire dans Docs Chain avec des transformations vérifiées.

Pile technologique (pourquoi + comment)

- **Go** - logique centrale d'événements et de diffusion (adaptée aux schémas linguistiques de chaque organisation).
- **Shell** - outils et scripts de configuration pour les opérateurs.
- **Code Claude (LLM)** - niveau d'inférence d'intention pour les messages en texte libre.
- **Adaptateurs de canaux Messenger** - diffusion multi-canaux des notifications.
- **Docs Chain** - pipeline de construction et de vérification de la documentation (Markdown→HTML/PDF/DOCX).
- **Sous-module Helix Constitution** - gouvernance et règles héritées.